



Short communication

Gaussian mixture learning via adaptive hierarchical clustering

Jichuan Li¹, Arye Nehorai^{1,*}

Department of Electrical and Systems Engineering, Washington University in St. Louis, St. Louis, MO 63130, United States

ARTICLE INFO

Article history:

Received 28 January 2018

Revised 10 April 2018

Accepted 11 April 2018

Available online 12 April 2018

Keywords:

Gaussian mixture learning
 Adaptive hierarchical clustering
 Adaptive splitting
 Expectation-maximization algorithm
 Distributed particle filtering

ABSTRACT

In signal processing, a large number of samples can be generated by a Monte Carlo method and then encoded as a Gaussian mixture model for compactness in computation, storage, and communication. With a large number of samples to learn from, the computational efficiency of Gaussian mixture learning becomes important. In this paper, we propose a new method of Gaussian mixture learning that works both accurately and efficiently for large datasets. The proposed method combines hierarchical clustering with the expectation-maximization algorithm, with hierarchical clustering providing an initial guess for the expectation-maximization algorithm. We also propose adaptive splitting for hierarchical clustering, which enhances the quality of the initial guess and thus improves both the accuracy and efficiency of the combination. We validate the performance of the proposed method in comparison with existing methods through numerical examples of Gaussian mixture learning and its application to distributed particle filtering.

© 2018 Elsevier B.V. All rights reserved.

1. Introduction

A Gaussian mixture model is a parametric approximation to a probability distribution via a weighted combination of Gaussian components [1]. It is often used to represent samples from an unknown probability distribution in a compact fashion. In signal processing, the number of samples can be large, for example, when the samples are intended to approximate a multi-dimensional probability distribution. This brings Gaussian mixture learning a challenge in its computational efficiency.

A traditional approach to Gaussian mixture learning is the expectation-maximization (EM) algorithm [2]. As an iterative optimization method, the EM algorithm can be slow, especially when the number of samples is large. Also, it is not adaptive in the number of Gaussian components in the mixture model, and the local maximum likelihood solution it provides depends heavily on the initial guess. Model selection criteria [3–5] can help the EM algorithm with adaptivity in the number of mixture components, by comparing different models returned by the EM algorithm when initialized with different numbers of components. However, given that the EM algorithm can be slow by itself, EM-based model selection approaches are often considerably expensive in computation. Sampling-based approaches [6–9] are adaptive, accurate, but often very slow when learning from a large number of samples. Hi-

erarchical clustering approaches [10,11] are often computationally efficient, but may suffer from accuracy issues. In summary, existing adaptive methods either compromise computational efficiency for approximation accuracy or the other way around.

In this paper, we propose a Gaussian mixture learning method that combines hierarchical clustering with the EM algorithm by feeding the output of hierarchical clustering to the EM algorithm as its initial guess. Also, we propose an adaptive hierarchical clustering approach to be used in the combination. The proposed hierarchical clustering approach is adaptive in two aspects. First, it determines the number of components in the Gaussian mixture adaptively based on data. Second, different from existing work, it recursively identifies and splits between potential Gaussian components, thus providing an informed initial guess for the EM algorithm. Thanks to adaptive splitting in hierarchical clustering, the informed initial guess is often close to a maximum likelihood estimate and thus needs fewer EM iterations, which makes the EM algorithm fast enough for large datasets. Also, the quality of the informed initial guess potentially improves the accuracy of the EM output. Therefore, we expect the proposed Gaussian mixture learning method to excel at both accuracy and efficiency for large datasets. We validate the proposed method on numerical examples of Gaussian mixture learning in comparison with existing methods. Also, we apply the proposed method to distributed particle filtering (DPF) [12], where Gaussian mixture learning for a large number of weighted samples is needed, and demonstrate the advantages of the proposed method over existing methods through numerical examples of distributed particle filtering. A preliminary

* Corresponding author.

E-mail addresses: jichuanli@wustl.edu (J. Li), nehorai@wustl.edu (A. Nehorai).¹ This work was supported by the AFOSR Grant No. FA9550-16-1-0386.

work on the application of adaptive Gaussian mixture learning to DPF can be found in [13], and in this paper we approach Gaussian mixture learning separately with a rigorous algorithmic design, a detailed analysis, and a collection of numerical examples from different perspectives and contexts.

The rest of the paper is organized as follows. Section 2 describes the problem. Section 3 proposes adaptive hierarchical clustering. Section 4 proposes a new Gaussian mixture learning method based on adaptive hierarchical clustering. Section 5 validates the proposed method through numerical examples. Section 6 concludes the paper.

2. Problem statement

We consider a dataset $\{\mathbf{x}^{(m)}, w^{(m)}\}_{m=1}^M$, where $\mathbf{x}^{(m)} \in \mathbb{R}^D$ is the m th sample in the dataset, $w^{(m)}$ is the weight of $\mathbf{x}^{(m)}$ with $w^{(m)} > 0$ and $\sum_{m=1}^M w^{(m)} = 1$, and M is the total number of samples in the dataset. We assume that the samples are generated to approximate an unknown probability distribution $p(\mathbf{x})$ as

$$p(\mathbf{x}) \approx \sum_{m=1}^M w^{(m)} \delta(\mathbf{x} - \mathbf{x}^{(m)}), \quad (1)$$

where δ is a multi-dimensional dirac delta function. Such a non-parametric approximation is often costly for computation, storage, and communication, and therefore when there is a constraint on any of them, it is sometimes preferable to approximate the probability distribution in a parametric fashion with a Gaussian mixture model as

$$p(\mathbf{x}) \approx \sum_{c=1}^C \alpha_c \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c), \quad (2)$$

where $\mathcal{N}$ indicates a (multivariate) Gaussian distribution, C is the total number of Gaussian components in the mixture, and α_c , $\boldsymbol{\mu}_c$, and $\boldsymbol{\Sigma}_c$ are, respectively, the weight, mean, and covariance matrix of the c th Gaussian component, with $\alpha_c > 0$ for $\forall c$ and $\sum_{c=1}^C \alpha_c = 1$.

The accuracy of a Gaussian mixture model is often evaluated by its log likelihood

$$L = \sum_{m=1}^M w^{(m)} \log \left[\sum_{c=1}^C \alpha_c \mathcal{N}(\mathbf{x}^{(m)}; \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \right], \quad (3)$$

but accuracy is not the only metric to optimize for a Gaussian mixture model. We can almost always improve accuracy by having more Gaussian components, but this contradicts with the compactness we want from a parametric representation. Thus, we often need to balance between accuracy and compactness for a Gaussian mixture model.

3. Adaptive hierarchical clustering

The first, also the most important, step of our Gaussian mixture learning method is adaptive hierarchical clustering, in which we recursively split a dataset into two in an adaptive fashion.

3.1. Dimension reduction

It is often computationally intensive to find an optimal way to split a dataset in a multi-dimensional space. To circumvent this challenge, we simplify the problem by projecting the samples in the dataset onto a single dimension, finding the optimal point at which to split the one-dimensional dataset, and then projecting the one-dimensional boundary back to the original high-dimensional space.

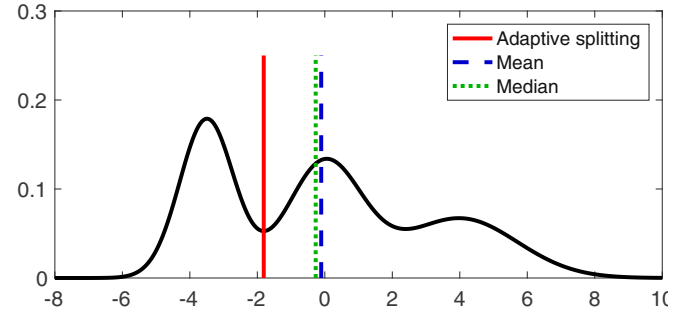


Fig. 1. An example of splitting at the mean, the median, and the deepest local minimizer.

Assuming that the optimal boundary is unlikely to align with the most variable direction, we project the weighted samples onto their principal components [14] $\{\psi_1, \psi_2, \dots, \psi_D\}$, indexed in descending order of variance after projection. For each $d \in \{1, 2, \dots, D\}$, we have weighted scalar samples $\{\psi_d^T \mathbf{x}^{(m)}, w^{(m)}\}_{m=1}^M$ and denote them as $\{x_{(d)}^{(m)}, w^{(m)}\}_{m=1}^M$. In the rest of this section, we introduce how to split these one-dimensional samples.

3.2. Kernel density estimation

In order to make an informed decision on how to split a one-dimensional dataset, we first learn the statistical distribution of the one-dimensional samples via weighted kernel density estimation (KDE) [15] as follows:

$$\hat{p}_s(x) = \sum_{m=1}^M w^{(m)} \phi_s(x, x_{(d)}^{(m)}), \quad (4)$$

where ϕ_s is a kernel function with a smoothing parameter s , and $\hat{p}_s$ is an estimate of the underlying distribution given s . Intending to learn a Gaussian mixture model, we naturally set ϕ_s as Gaussian kernel, and hence s can be interpreted as the standard deviation of the Gaussian kernel. By default, we determine s adaptively based on an approximation of the minimum mean integrated squared error rule [15]:

$$s = \sigma \left(\frac{4}{3M} \right)^{1/5}, \quad (5)$$

where σ is the standard deviation of the underlying distribution and substituted with a robust estimate [15]:

$$\hat{\sigma} = \text{median} \left\{ \left| x_{(d)}^{(j)} - \text{median} \{ x_{(d)}^{(i)}, w^{(i)} \}_{i=1}^{j=M} \right|, w^{(j)} \right\}_{j=1}^{j=M} / 0.6745. \quad (6)$$

3.3. Adaptive splitting

Given the univariate distribution $\hat{p}_s$ in (4), in order to split the projected dataset into two, we only need to find a scalar boundary point along the projected direction. Most existing work choose the mean or median as the splitting point [10,11], which might not be a good strategy for Gaussian mixture learning, because the mean or median is not guaranteed to fall between potential Gaussian components, and splitting at the mean or median might undesirably break a Gaussian component apart, as shown in Fig. 1. To avoid this drawback, we split the dataset using an alternative strategy that makes full use of the sample distribution $\hat{p}_s$ learned in (4).

Based on the fact that the probability density of a point decreases as it deviates from the center of a Gaussian component, we assume that a point with a local minimum density is likely to

e on the boundary between Gaussian components, and thus split the dataset at the deepest local minimizer, as shown in Fig. 1.

To find the deepest local minimizer, we evaluate $\hat{p}_s$ on a one-dimensional grid of G points. If there is no local minimizer found, we claim the entire cluster as a single component and do not split any further; otherwise, we look for the deepest local minimizer (the depth of a local minimizer is defined as the minimum vertical distance that a point has to ascend on either side of the minimizer before reaching a point with an equal or lower value) and consider it as the splitting point. Samples with projected values lower than the splitting point are grouped as one child dataset, and those with projected values greater than the splitting point are grouped as the other child dataset.

4. Adaptive hierarchical tree

Based on adaptive splitting, we build an adaptive hierarchical tree to represent the dataset in a hierarchical fashion. We consider the whole dataset as the root of the tree and recursively build the tree from it. For each node in the tree, if the number of samples is below a given threshold, we consider the node as a leaf and do not split it (for simplicity we set the threshold to 10 by default, assuming that we do not have enough confidence in the statistical distribution learned from fewer than 10 samples to make a splitting decision, although the threshold can be selected in a more adaptive way). Otherwise, we project the samples onto a principal component, from the first to the last, and perform adaptive splitting. If a qualified local minimizer is found for a principal component, we split the current node at the minimizer without considering the remaining principal components; otherwise, we consider the current node as a leaf and do not further split it. We obtain a Gaussian mixture model from the leaves of the tree, with each leaf corresponding to a Gaussian component and the sum of sample weights on the leaf corresponding to the weight of the Gaussian component in the mixture. The adaptive hierarchical clustering algorithm is summarized in Algorithm 1.

Algorithm 1 Adaptive hierarchical clustering.

```

1: procedure AHC( $\{\mathbf{x}^{(m)}, w^{(m)}\}_{m=1}^M$ )
2:   if  $M < 10$  then
3:     return {GAUSSIAN( $\{\mathbf{x}^{(m)}, w^{(m)}\}_{m=1}^M$ )}
4:   end if
5:    $\{\psi_1, \psi_2, \dots, \psi_D\} = \text{WEIGHTEDPCA}(\{\mathbf{x}^{(m)}, w^{(m)}\}_{m=1}^M)$ 
6:   for  $d = 1$  to  $D$  do
7:     for  $m = 1$  to  $M$  do
8:        $x^{(m)} = \psi_d \mathbf{x}^{(m)}$ 
9:     end for
10:     $\hat{p}_s = \text{WEIGHTEDKDE}(\{x^{(m)}, w^{(m)}\}_{m=1}^M)$ 
11:     $S = \text{FINDLOCALMINIMIZER}(\hat{p}_s)$ 
12:    if  $S \neq \emptyset$  then
13:       $x^* = \arg \max_{x \in S} \text{DEPTH}(x)$ 
14:       $C_1 = \text{AHC}(\{x^{(m)}, w^{(m)}\}_{x^{(m)} \leq x^*})$ 
15:       $C_2 = \text{AHC}(\{x^{(m)}, w^{(m)}\}_{x^{(m)} > x^*})$ 
16:      return  $C_1 \cup C_2$ 
17:    end if
18:  end for
19:  return {GAUSSIAN( $\{\mathbf{x}^{(m)}, w^{(m)}\}_{m=1}^M$ )}
20: end procedure

```

5. Computational complexity

In each recursive step of adaptive hierarchical clustering, we perform weighted principal component analysis once at the cost of $O(D^3)$ and perform adaptive splitting no more than D times.

Within adaptive splitting, KDE takes $O(GM)$, and finding the splitting point takes $O(G)$. Therefore, each recursive step costs $O(D^3 + DGM + DG) = O(DGM)$, assuming $D \ll G$ and $D \ll M$. In comparison, a KD tree, which splits at the mean, has a lower complexity $O(DM)$ than adaptive hierarchical clustering in each recursive step, but adaptive hierarchical clustering incurs less computation in total, for the following reasons. First, unlike a KD tree, which recursively splits a cluster until it is too small to split, adaptive hierarchical clustering stops splitting much earlier when it finds out that a node has only one component in it. In other words, a KD tree is often overbuilt with unnecessary nodes, while adaptive hierarchical clustering stops early to avoid building unnecessary nodes. Second, unlike a KD tree, which needs to search the entire tree for an optimal combination of nodes as Gaussian components, adaptive hierarchical clustering directly obtains Gaussian components from its leaves with no need for an extra search step. Furthermore, given that a KD tree is often overbuilt, the tree search can be computationally intensive. Last but not least, unlike a KD tree, which is not aware of clusters or Gaussian components during tree building, adaptive hierarchical clustering actively identifies potential Gaussian components all through the tree building process, and thus provides a more informed initial guess to the EM algorithm, which results in fewer EM iterations. We will validate these analyses and claims with numerical examples in the Section 5.

4. Gaussian mixture learning

The Gaussian mixture learning method, as summarized in Algorithm 2, simply runs adaptive hierarchical clustering and feeds

Algorithm 2 Gaussian mixture learning.

```

1: procedure GML( $\{\mathbf{x}^{(m)}, w^{(m)}\}_{m=1}^M$ )
2:    $\text{GMM} = \text{AHC}(\{\mathbf{x}^{(m)}, w^{(m)}\}_{m=1}^M)$ 
3:    $\text{GMM}^* = \text{EM}(\text{GMM}, \{\mathbf{x}^{(m)}, w^{(m)}\}_{m=1}^M)$ 
4:   return  $\text{GMM}^*$ 
5: end procedure

```

its output to the EM algorithm as its initial guess. The EM algorithm that learns a Gaussian mixture model from weighted samples can be found in [16]. The EM algorithm costs $O(CD^2IM)$, where I is the number of EM iterations.

5. Numerical examples

We tested the proposed Gaussian mixture learning method on numerical examples of Gaussian mixture learning and its application to DPF. We compared the proposed method (AHC-EM) with (1) AHC without EM, (2) KD tree [17], which performs hierarchical clustering by splitting at the mean, (3) the EM algorithm, with the initial guess being one of multiple random initializations that results in the highest likelihood after a few EM iterations, (4) KD-EM, the combination of KD tree with the EM algorithm, and (5) variational inference for Dirichlet process mixtures (VIDPM) [9] based on sampling. We also tested other methods [3–8], but they were prohibitively slow for a large number of samples, so that we were unable to obtain any results to present in this section.

5.1. Gaussian mixture learning

We tested Gaussian mixture learning methods on samples randomly generated from Gaussian mixtures of 2, 4, and 6 components. For each Gaussian component $\alpha \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$, we randomly generated α as a random number from a uniform distribution $\mathcal{U}(0.2, 0.8)$ followed by normalization, $\boldsymbol{\mu}$ as $[\mu_1; \mu_2]$ with μ_1 and μ_2 independent and identically distributed (i.i.d.) as $\mathcal{U}(-20, 20)$,

Table 1

Gaussian mixture learning for 2-component Gaussian mixtures.

	AHC-EM	AHC	KD-EM	KD	EM	VIDPM
Log-likelihood	−3.7390	−3.7416	−3.7390	−3.7830	−3.7390	−3.7775
Runtime / s	0.0560	0.0348	0.2946	0.2688	0.5926	0.1079
Components	2.06	2.06	2.05	2.05	2.00	2.03
EM iterations	7.10	–	7.57	–	340.43	–

Table 2

Gaussian mixture learning for 4-component Gaussian mixtures.

	AHC-EM	AHC	KD-EM	KD	EM	VIDPM
Log-likelihood	−4.5146	−4.5317	−4.6089	−4.7771	−4.5258	−4.4953
Runtime / s	0.0944	0.0537	0.3345	0.2926	0.3106	0.2028
Components	3.87	3.87	3.30	3.30	4.00	3.93
EM iterations	13.29	–	14.63	–	59.01	–

Table 3

Gaussian mixture learning for 6-component Gaussian mixtures.

	AHC-EM	AHC	KD-EM	KD	EM	VIDPM
Log-likelihood	−4.8814	−4.9061	−5.1502	−5.3953	−4.9736	−4.9527
Runtime / s	0.2680	0.0828	0.4322	0.3491	0.5084	0.3749
Components	6.22	6.22	4.16	4.16	6.00	5.78
EM iterations	39.20	–	28.78	–	48.09	–

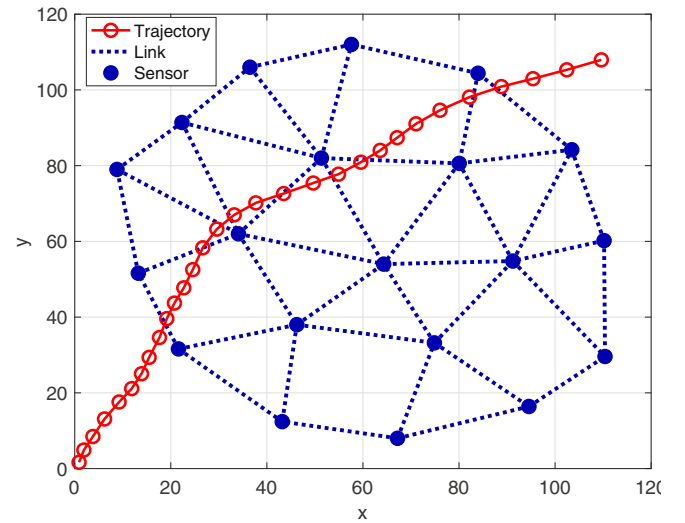
and Σ as $[\sigma_{11}, \sigma_{12}; \sigma_{21}, \sigma_{22}]$ with σ_{11} and σ_{22} i.i.d. as $\mathcal{U}(0, 5)$ and $\sigma_{12} = \sigma_{21}$ distributed as $\mathcal{U}(-\sqrt{\sigma_{11}\sigma_{22}}, \sqrt{\sigma_{11}\sigma_{22}})$. For each Gaussian mixture, we randomly generated 5000 samples for training and another 5000 samples for testing, so that the weight of each sample was 1/5000. Because the EM algorithm is unable to figure out the number of components in the Gaussian mixture, we assumed the ground truth of the number of components to be known in its initial guess. For KDE in AHC, we used 50 grid points.

Tables 1–3 show the experiment metrics averaged over 100 repeated runs. Among all the compared methods, AHC-EM had the highest log-likelihood and was thus the most accurate. AHC alone was more accurate than KD, and sometimes even more accurate than KD-EM, which demonstrates the benefit of adaptive splitting in hierarchical clustering and shows the high quality of an initial guess that AHC can provide for the EM algorithm. AHC-EM and KD-EM were more accurate than AHC and KD, respectively, which shows the value that the EM algorithm adds to Gaussian mixture learning in the combination with hierarchical clustering. Based on the comparison of runtimes, AHC-EM was the fastest after AHC, thanks to the efficiency of AHC and the reduced EM iterations. Moreover, most of the methods returned a reasonable number of components, although KD tended to underestimate it.

5.2. Distributed particle filtering

In practice, samples that a Gaussian mixture model is learned from can come from a complicated distribution, instead of a regular Gaussian mixture. Hence, we also tested Gaussian mixture learning on its application to DPF [12], where each sensor learns a Gaussian mixture model from the importance samples of its posterior estimate (the design of DPF in [12] is independent of the choice of a Gaussian mixture learning approach, and thus the numerical example of DPF generalizes beyond this specific DPF approach). More specifically, we considered a network of 20 sensors tracking a single moving target, as shown in Fig. 2. The target followed a constant velocity motion model [18] with additive Gaussian noise on a two-dimensional plane, according to the following state transition equation

$$\mathbf{x}_n = \mathbf{D} \cdot \mathbf{x}_{n-1} + \mathbf{u}_n, \quad (7)$$

**Fig. 2.** Sensor network and target trajectory.

where

$$\mathbf{x}_n = \begin{bmatrix} x_{n,1} \\ x_{n,2} \\ \dot{x}_{n,1} \\ \dot{x}_{n,2} \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{R} = \begin{bmatrix} \frac{1}{6} & 0 & \frac{1}{4} & 0 \\ 0 & \frac{1}{6} & 0 & \frac{1}{4} \\ \frac{1}{4} & 0 & 1 & 0 \\ 0 & \frac{1}{4} & 0 & 1 \end{bmatrix},$$

$$\mathbf{x}_1 \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \mathbf{R}\right), \quad \mathbf{u}_n \sim \mathcal{N}(\mathbf{0}, \mathbf{R}).$$

The k th sensor, located at $(l_{k,1}, l_{k,2})$, measures its distance to the target, according to the observation equation

$$y_{n,k} = \sqrt{(x_{n,1} - l_{k,1})^2 + (x_{n,2} - l_{k,2})^2} + v_{n,k}, \quad (8)$$

where $v_{n,k} \sim \mathcal{N}(0, 0.7)$. Unless otherwise stated, we used 600 particles for local particle filtering at each sensor, 15,000 samples for importance sampling in distributed fusion, 1000 grid points for KDE, and 40 iterations for average consensus.

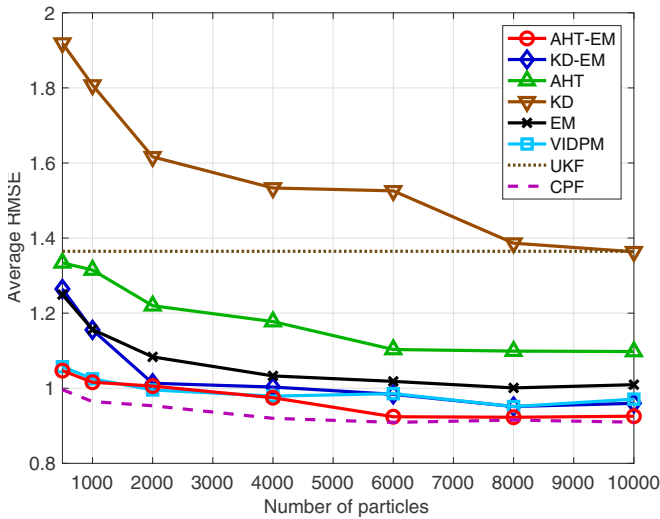


Fig. 3. Average root-mean-square error (RMSE) as a function of the number of particles per sensor.

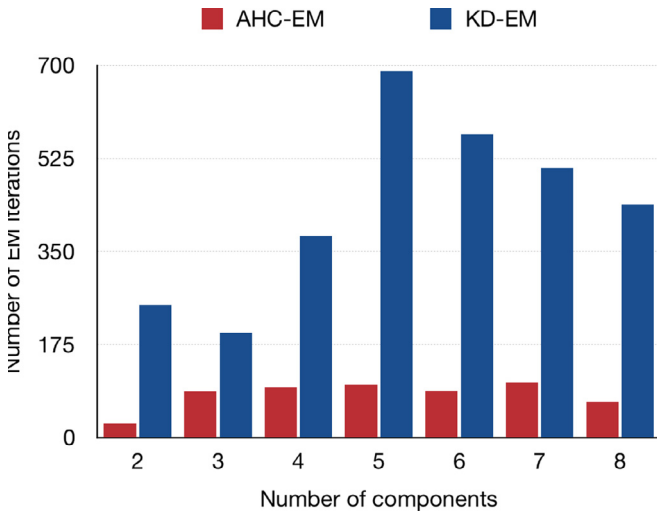


Fig. 4. The number of EM iterations as a function of the number of components in a Gaussian mixture model.

Fig. 3 compares the estimation errors of DPF using different Gaussian mixture learning methods, averaged over repeated experiments, with centralized particle filtering (CPF) setting the benchmark. As expected, the error of DPF decreased as the number of particles increased. Among all the Gaussian mixture learning methods, the proposed method, AHC-EM, yielded the lowest error, the closest to that of CPF. AHC-EM and AHC resulted in lower errors than KD-EM and KD, respectively, which confirms the advantage of adaptive splitting in hierarchical clustering. AHC-EM and KD-EM resulted in lower errors than AHC and KD, respectively, which confirms the value that the EM algorithm adds to the combination. M, initialized with 4 components, outperformed AHC and KD, but was outperformed by AHC-EM, KD-EM, and VIDPM, which verifies our previous claims that hierarchical clustering alone might not be sufficiently accurate and that the EM algorithm benefits from an informed initial guess.

Fig. 4 compares the average numbers of EM iterations needed by AHC-EM and KD-EM. We can see that AHC-EM needed fewer EM iterations than KD-EM did, which indicates that AHC-EM is generally faster than KD-EM in the EM step. Also, a small number of EM iterations means that the initial guess is close to a

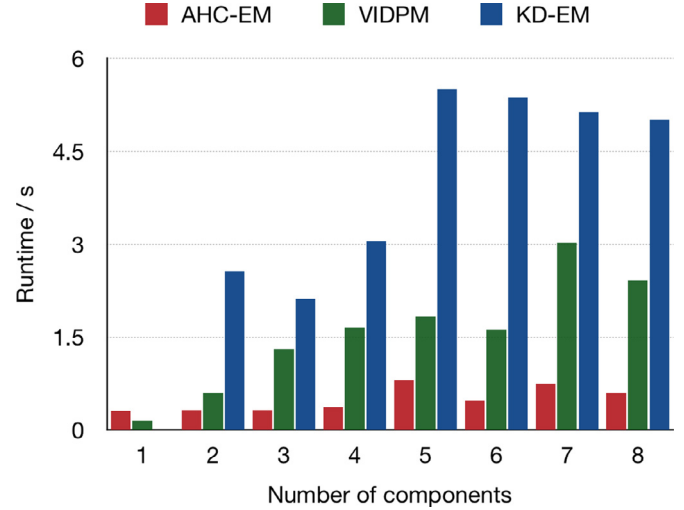


Fig. 5. The average runtime as a function of the number of components in a Gaussian mixture model.

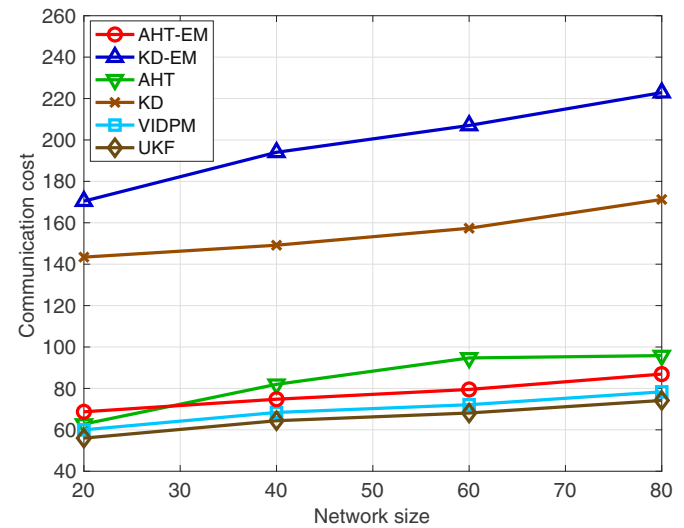


Fig. 6. Communication cost per sensor per consensus iteration as a function of the network size.

maximum likelihood solution, and thus implies that AHC provides a more informed EM initial guess than KD does.

Fig. 5 compares the average runtimes of different methods tested under the same computing environment within DPF. The runtimes of AHC-EM were much lower than those of KD-EM, which agrees with Fig. 4; the runtimes of AHC-EM were lower than those of VIDPM, except for single-component Gaussian mixtures. Note that the runtimes of both KD-EM and VIDPM increased rapidly as the number of components grew, while those of AHC-EM almost remained at the same level as the number of components changed, which demonstrates the scalability of AHC-EM.

Fig. 6 compares the communication cost of DPF using different Gaussian mixture learning methods under different network sizes. The communication cost is evaluated as the amount of numbers used to represent the Gaussian mixture models exchanged across the network per sensor per consensus iteration. We can see from Fig. 6 that the communication cost slightly increased as the network size grew. Also, the communication cost of AHC-EM was slightly higher than that of VIDPM, slightly lower than that of AHC, and much lower than those of KD and KD-EM. Since the communication cost is approximately proportional to the average number

of components generated, the comparison reflects how much each method tended to underestimate or overestimate the number of components.

6. Conclusion

In this paper, we proposed a Gaussian mixture learning method, based on the combination of hierarchical clustering and the EM algorithm. We proposed adaptive splitting and designed adaptive hierarchical clustering based on it. Adaptive hierarchical clustering improves the computational efficiency of hierarchical clustering, enhances the quality of the initial guess to the EM algorithm, and reduces the number of EM iterations needed in the combination with the EM algorithm. Based on numerical examples of Gaussian mixture learning and its application to DPF, we showed that the proposed method is both accurate and efficient, even with a large number of samples to learn from.

As an unsupervised learning approach, the proposed Gaussian mixture learning method applies to not only DPF, but also many other problems, including data fusion [16], clustering [19], and image processing [20], where Gaussian mixture learning is needed, potentially for a large number of components. In future work, we will explore these interesting applications and investigate Gaussian mixture learning for a large number of components.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at doi:[10.1016/j.sigpro.2018.04.013](https://doi.org/10.1016/j.sigpro.2018.04.013).

References

- [1] C.M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2007.
- [2] A.P. Dempster, N.M. Laird, D.B. Rubin, Maximum likelihood from incomplete data via the EM algorithm, *J. R. Stat. Soc. Series B* 39 (1) (1977) 1–38.
- [3] C. Olivier, F. Jouzel, A. Matouat, Choice of the number of component clusters in mixture models by information criteria, in: *Proc. Vision Interface*, 199 pp. 74–81.
- [4] A. Dasgupta, A.E. Raftery, Detecting features in spatial point processes with clutter via model-based clustering, *J. Am. Stat. Assoc.* 93 (441) (1998) 294–30.
- [5] C. Fraley, A.E. Raftery, How many clusters? which clustering method? answer via model-based cluster analysis, *Comput. J.* 41 (8) (1998) 578–588.
- [6] C.E. Rasmussen, The infinite Gaussian mixture model, in: *NIPS*, 12, 199 pp. 554–560.
- [7] M. Stephens, Bayesian analysis of mixture models with an unknown number of components—an alternative to reversible jump methods, *Ann. Stat.* (2001) 40–74.
- [8] M.A.T. Figueiredo, A.K. Jain, Unsupervised learning of finite mixture model, *IEEE Trans. Pattern Anal. Mach. Intell.* 24 (3) (2002) 381–396.
- [9] D.M. Blei, M.I. Jordan, et al., Variational inference for dirichlet process mixtures, *Bayesian Anal.* 1 (1) (2006) 121–144.
- [10] D. Boley, Principal direction divisive partitioning, *Data Min. Knowl. Discov.* (4) (1998) 325–344.
- [11] A.T. Ihler, J.W. Fisher III, A.S. Willsky, Particle filtering under communication constraints, in: *IEEE 13th Workshop on Statistical Signal Processing*, Bordeaux France, 2005, pp. 89–94.
- [12] J. Li, A. Nehorai, Distributed particle filtering via optimal fusion of Gaussian mixtures, *IEEE Trans. Signal Inf. Process. Netw.* PP (99) (2017).
- [13] J. Li, A. Nehorai, Adaptive Gaussian mixture learning in distributed particle filtering, in: *2015 6th International Workshop on Computational Advances in Multi-Sensor Adaptive Processing (CAMSAP)*, Cancún, Mexico, 201 pp. 221–224.
- [14] Z. Fan, E. Liu, B. Xu, Weighted principal component analysis, in: *International Conference on Artificial Intelligence and Computational Intelligence*, Springer, 2011, pp. 569–574.
- [15] A.W. Bowman, A. Azzalini, *Applied Smoothing Techniques for Data Analysis*, Oxford University Press, 1997.
- [16] N.R. Ahmed, M. Campbell, Fast consistent Chernoff fusion of Gaussian mixtures for ad hoc sensor networks, *IEEE Trans. Signal Process.* 60 (12) (2012) 6739–6745.
- [17] A. Moore, A tutorial on kd-trees, Technical report, University of Cambridge Computer Laboratory, 1991.
- [18] Y. Bar-Shalom, P.K. Willett, X. Tian, *Tracking and Data Fusion: A Handbook of Algorithms*, YBS Publishing, 2011.
- [19] K.J. Cios, R.W. Swiniarski, W. Pedrycz, L.A. Kurgan, Unsupervised Learning: Clustering, in: *Data Mining*, Springer, 2007, pp. 257–288.
- [20] H. Permuter, J. Francos, I. Jermyn, A study of Gaussian mixture models of color and texture features for image classification and segmentation, *Pattern Recognition* 39 (4) (2006) 695–706.